Análisis

Integrantes:
Cristian Aliaga
Nicolas Urbina
John Rojas

Fecha:
30 de octubre de 2026

1. Introducción

Las tres implementaciones utilizan WebSocket para establecer una comunicación bidireccional y persistente entre el navegador y el servidor. La diferencia principal está en el nivel de funcionalidad y control incorporado en cada versión. La primera demuestra el funcionamiento básico, la segunda separa la entrega web de la comunicación WebSocket y la tercera agrega identificación de usuarios, presencia y detección de desconexiones abruptas.

2. Primera implementación: comunicación directa

Esta versión utiliza `ChatWebSocketServer`, `Main` e `index.html`. El servidor se ejecuta en el endpoint `/ws/chat` y mantiene las sesiones activas en un conjunto sincronizado.

Funcionamiento lógico

- El cliente abre una conexión WebSocket.
- La sesión se agrega al conjunto de sesiones activas.
- Los mensajes recibidos se retransmiten a todos los clientes.
- Las conexiones y desconexiones se notifican mediante mensajes de sistema.

Comunicación

Los mensajes se envían como texto plano, por ejemplo:

```
[ID_Sesión]: mensaje
```

No existen nombres de usuario, mensajes privados ni un formato formal para los datos. El servidor envía los mensajes usando `BasicRemote.sendText`.

Ventajas y limitaciones

- **Ventajas:** es simple, fácil de comprender y permite demostrar los eventos principales de WebSocket.
- **Limitaciones:** identifica mediante el ID interno de la sesión, todos los mensajes son públicos y no detecta específicamente una desconexión abrupta.

3. Segunda implementación: HTTP y WebSocket separados

Esta versión utiliza `Main2` e `index2.html`. Se ejecutan dos servicios independientes: un servidor HTTP Grizzly en el puerto 8080 y un servidor WebSocket Tyrus en el puerto 8081.

Servicio	Puerto	Responsabilidad
Servidor HTTP Grizzly	8080	Entrega HTML, CSS y JavaScript.
Servidor WebSocket Tyrus	8081	Gestiona conexiones y mensajes en tiempo real.

La interfaz se carga desde `http://localhost:8080/index2.html` y luego se conecta a `ws://localhost:8081/ws/chat`.

Importancia estructural

A diferencia de la primera versión, la entrega de archivos web queda separada de la lógica de comunicación. Esto representa una arquitectura más organizada y cercana a una aplicación distribuida real.

- **Ventajas:** separación de responsabilidades, mayor flexibilidad y posibilidad de escalar los servicios de forma independiente.
- **Limitaciones:** aumenta la configuración, requiere administrar dos puertos y mantiene la comunicación como texto plano.

4. Tercera implementación: usuarios, presencia y heartbeat

Esta versión utiliza `ChatWebSocketServer3`, `Main3` e `index3.html`. Mantiene la separación HTTP/WebSocket y agrega mecanismos para identificar usuarios y comprobar que las conexiones siguen activas.

Identificación y lista de usuarios

El nombre se envía como parámetro de la URL:

```
ws://localhost:8081/ws/chat?nombre=Juan
```

El servidor asocia cada objeto `Session` con un nombre mediante un `ConcurrentHashMap`. También difunde mensajes JSON con la lista de usuarios:

```
{"tipo":"LISTA_USUARIOS","usuarios":["Juan","Ana"]}
```

Heartbeat PING/PONG

1. Cada 10 segundos el servidor envía un frame `PING`.
2. El cliente responde con `PONG`.
3. El servidor registra la hora del último `PONG`.
4. Si pasan 30 segundos sin respuesta, se cierra la sesión.

5. Se notifica a los demás usuarios la pérdida de conexión.

- **Ventajas:** usuarios identificados, lista de participantes, detección de fallos y mejor experiencia de uso.
- **Limitaciones:** requiere mayor control de concurrencia y la lista JSON se construye manualmente; los mensajes de chat siguen siendo texto plano.

5. Comparación general

Característica	Implementación 1	Implementación 2	Implementación 3
Comunicación	WebSocket directo	HTTP + WebSocket separados	HTTP + WebSocket separados
Puertos	8080	8080 y 8081	8080 y 8081
Identificación	ID de sesión	ID de sesión	Nombre de usuario
Difusión pública	Sí	Sí	Sí
Lista de usuarios	No	No	Sí
Mensajes estructurados	No	No	Parcialmente, mediante JSON
Heartbeat	No	No	Sí
Desconexión abrupta	No controlada específicamente	No controlada específicamente	Detectada mediante timeout
Complejidad	Baja	Media	Alta

6. Diferencias en la estructura lógica

La evolución de las implementaciones puede resumirse de la siguiente manera:

```
Chat básico
    ↓
Separación entre servidor HTTP y servidor WebSocket
    ↓
Usuarios identificados + presencia + control de fallos
```

La primera versión concentra toda la responsabilidad en un endpoint sencillo. La segunda separa la publicación de recursos web de la comunicación en tiempo real. La tercera agrega administración de usuarios y supervisión periódica de las conexiones.

7. Diferencias en la comunicación

En las dos primeras versiones, los mensajes son cadenas simples. En la tercera se agregan mensajes de control en JSON para actualizar la lista de usuarios. Además, se utilizan los frames de control propios de WebSocket, `PING` y `PONG`, para verificar la disponibilidad de los clientes.

> Por lo tanto, la tercera implementación transmite tanto información de chat como información de estado de la conexión.

8. Conclusión

La primera implementación es adecuada para demostrar el funcionamiento fundamental de WebSocket. La segunda mejora la organización del sistema al separar el servidor HTTP del servidor WebSocket. La tercera es la más completa porque incorpora identificación de usuarios, presencia en línea y detección de desconexiones abruptas mediante heartbeat.

En consecuencia, las implementaciones muestran una evolución desde una solución demostrativa y sencilla hacia una arquitectura más cercana a una aplicación real. Esta evolución también implica mayor complejidad, especialmente en la gestión concurrente de sesiones y en la estructuración de los mensajes.